

二分查找

二分查找是一种在有序数组中查找特定元素的高效算法。它通过不断将搜索区间缩小一半，逐步逼近目标元素，从而将查找的时间复杂度从线性的 $O(n)$ 降低到对数级别的 $O(\log n)$ 。

先来玩一个经典的猜数字的小游戏。

猜数字游戏

通过这个游戏我们发现，如果每次都猜当前能够选取的数字的中间值，这个策略是最优的。

不妨想想对于 1 – 100 之间的任何一个数，最多几次一定能够猜到？

答案是 7 次!

第一次我们猜区间的中间值 50。如果猜中，查找立即结束；如果没有猜中，就会得到“猜大了”或“猜小了”的提示。由于数字的大小关系具有单调性，我们可以据此排除不可能的一半区间，只在剩余的一半中继续查找。

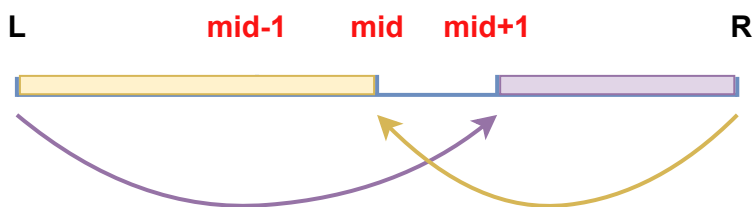
重复这一过程，每次都猜当前区间的中间值，待查找范围就会不断缩小。对于 1 到 100 这 100 个数，由于

$$2^6 = 64 < 100 < 128 = 2^7。$$

因此二分查找在最坏情况下最多需要猜 7 次，就一定能够找到答案。

经典模型1：查询第一个大于等于 x 的位置

按照二分的策略，首先考虑中间元素，如果中间元素大于等于 x ，那么说明右边的元素不可能作为第一个大于等于 x 的位置，所以舍弃右边。反之，则说明左边的元素不可能大于等于 x ，舍弃左边。



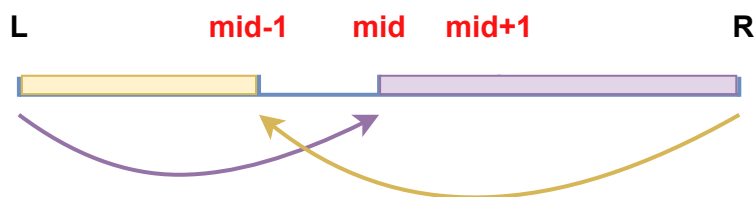
```
1  int l = 1, r = n;
2  while (l < r) {
3      int mid = l + r >> 1;
4      if (a[mid] >= x) r = mid; // 舍弃右边
5      else l = mid + 1; // 舍弃左边
6  }
7  cout << l << "\n";
```

例题: #P456. 【例83.2】二分查找

思路: 我们直接二分查找第一个大于等于 x 的元素, 然后判断该元素是否等于 x 即可。

经典模型2: 查询最后一个小于等于 x 的位置

按照二分的策略, 首先考虑中间元素, 如果中间元素小于等于 x , 那么说明左边的元素不可能作为最后一个小于等于 x 的位置, 所以舍弃左边。反之, 则说明右边的元素不可能小于等于 x , 舍弃右边。



```
1  int l = 1, r = n;
2  while (l < r) {
3      int mid = l + r + 1 >> 1; // 避免死循环
4      if (a[mid] ≤ x) l = mid;
5      else r = mid - 1;
6  }
7  cout << l << "\n";
```

例题: #P463. 练83.5 二分查找2

思路: 我们直接二分查找最后一个小于等于 x 的元素, 然后判断该元素是否等于 x 即可。

二分查找模板

```
1  // 区间[l, r]被划分成[l, mid]和[mid + 1, r]时使用:
2  int l = 1, r = n;
3  while (l < r) {
4      int mid = l + r >> 1;
5      if (check(mid)) r = mid; // 舍弃右边
6      else l = mid + 1; // 舍弃左边
7  }
8  cout << l << "\n";
```

```

1 // 区间[l, r]被划分成[l, mid - 1]和[mid, r]时使用:
2 int l = 1, r = n;
3 while (l < r) {
4     int mid = l + r + 1 >> 1; //避免死循环
5     if (check(mid)) l = mid;
6     else r = mid - 1;
7 }
8 cout << l << "\n";

```

补充内容

实数二分

```

1 //实数二分
2 bool check(double x) { /* ... */ } // 检查x是否满足某种性质
3
4 double bsearch_3(double l, double r)
5 {
6     const double eps = 1e-6; // eps 表示精度，取决于题目对精度的要求
7     while (r - l > eps)
8     {
9         double mid = (l + r) / 2;
10        if (check(mid)) r = mid;
11        else l = mid;
12    }
13    return l;
14 }

```

STL

数组 a 中, $0 \sim n$ 里第一个大于等于 m 的指针

```
lower_bound(a, a+n, m);
```

数组 a 中, $0 \sim n$ 里第一个大于等于 m 的位置

```
int pos=lower_bound(a, a+n, m)-a;
```

数组 a 中, $0 \sim n$ 里第一个大于 m 的指针

```
upper_bound(a, a+n, m);
```

数组 a 中, $0 \sim n$ 里第一个大于 m 的位置

```
int pos=upper_bound(a, a+n, m)-a;
```

例题： LG#P1102. A-B 数对

思路：

我们需要找有多少个 (A, B) 满足 $A - B = C$ 。

朴素做法：两重循环枚举 A 和 B ，统计满足条件的数量。

在 **枚举法** 中我们学习过，对于这种情况，我们可以只枚举一个未知变量。求解出另外一个变量，然后验证合法性即可。那么，对于本题，我们可以枚举 B ，那么 $A = B + C$ 。我们直接统计 $B + C$ 出现了多少次即可。

于是题目转换为：

如何求解一个数 x 在数组中出现了多少次。

容易想到使用 **数组计数**。本题数据范围较大，可以使用 `map` 代替数组。

参考代码：

```
1  #include<bits/stdc++.h>
2
3  using namespace std;
4
5  using i64=long long;
6  void Showball(){
7      int n,c;
8      cin>>n>>c;
9      vector<int> a(n);
10     map<i64,i64> mp;
11     for(int i=0;i<n;i++){
12         cin>>a[i];
13         mp[a[i]]++;
14     }
15     i64 ans=0;
16     for(int i=0;i<n;i++){
17         ans+=mp[a[i]+c];
18     }
19     cout<<ans;
20
21 }
22 int main(){
23     ios::sync_with_stdio(false);
24     cin.tie(nullptr);
25     Showball();
26     return 0;
27 }
```

对于本题，我们还可以使用二分查找来求解。具体地，我们可以找到第一个大于等于 x 的位置 L 以及第一个大于 x 的位置 R 。那么 x 的出现次数为 $R - L$ 。

注意：二分前需要保证单调性，因此需要先对数组进行排序。并且需要注意二分的范围。

参考代码：

```

1  #include<bits/stdc++.h>
2
3  using namespace std;
4
5  using i64=long long;
6
7  const int N = 2e5 + 10;
8  int a[N];
9  void Showball(){
10     int n,c;
11     cin>>n>>c;
12     for(int i=0;i<n;i++) cin>>a[i];
13     sort(a,a+n);
14     i64 ans=0;
15     for(int i=0;i<n;i++){
16         int l=0,r=n,b=a[i];
17         while(l<r){
18             int mid=l+r>>1;
19             if(a[mid]>=b+c) r=mid;
20             else l=mid+1;
21         }
22         int L=l;
23         l=0,r=n;
24         while(l<r){
25             int mid=l+r>>1;
26             if(a[mid]>b+c) r=mid;
27             else l=mid+1;
28         }
29         ans+=l-L;
30     }
31     cout<<ans<<"\n";
32 }
33 int main(){
34     ios::sync_with_stdio(false);
35     cin.tie(nullptr);
36     int t=1;
37     //cin>>t;
38
39     while(t--){
40         Showball();
41     }
42
43     return 0;
44 }

```

也可以使用 *STL* 更方便书写。

```

1  #include<bits/stdc++.h>
2
3  using namespace std;
4
5  using i64=long long;
6
7  const int N = 2e5 + 10;
8  int a[N];

```

```
9 void Showball(){
10     int n,c;
11     cin>>n>>c;
12     for(int i=0;i<n;i++) cin>>a[i];
13     sort(a,a+n);
14     i64 ans=0;
15     for(int i=0;i<n;i++){
16         int b=a[i];
17         int l=lower_bound(a,a+n,b+c)-a;
18         int r=upper_bound(a,a+n,b+c)-a;
19         ans+=r-l;
20     }
21     cout<<ans<<"\n";
22 }
23 int main(){
24     ios::sync_with_stdio(false);
25     cin.tie(nullptr);
26     int t=1;
27     //cin>>t;
28
29     while(t--){
30         Showball();
31     }
32
33     return 0;
34 }
```